

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓 名：张序鸿

学 院：竺可桢学院

专 业：计算机科学与技术

学 号：3230103265

指导教师：杨莹春

2024 年 12 月 23 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: 缓存 Cache 设计

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 马致远

实验地点: 东 4-509 实验日期: 2024 年 12 月 18 日

一、实验目的和要求

- 1.理解高速缓存的基本概念和作用
- 2.掌握缓存的组织结构和映射方式、替换策略
- 3.理解缓存的工作原理、命中率和一致性问题
- 4.设计缓存的控制器模块和存储模块

二、实验内容和原理

内容:

任务一: 数据缓存模块的设计

- Two-way set associate cache (两路组相联 Cache)

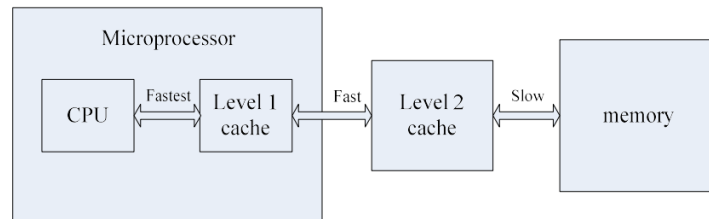
任务二: 数据缓存模块的仿真验证

原理：

2.1 Cache 基本概念

Cache--高速缓冲存储器,位于 CPU 与内存之间,是一种特殊的存储器子系统。

目前比较常见的是两极 cache 结构,即 cache 系统由 一级高速缓存 L1 cache 和二级高速缓存 L2 cache 组成, L1 cache 通常又分为数据 cache (D-Cache) 和指令 cache (I-Cache), 它们分别用来存放数据和处理这些数据的指令



为什么会需要 Cache? 电脑的内存是以系统总线的时钟频率工作的,也就是 CPU 的外频。但是, CPU 的工作频率(主频)是外频与倍频因子的乘积。因此,内存的工作频率就远低于 CPU 的工作频率了,导致的结果是: CPU 在执行完一条指令后,常常需要等待一些时间才能再次访问内存,极大降低了 CPU 工作效率。

此时 Cache 就诞生了,用来提高 CPU 对存储器的访问速度。

2.2 Cache 工作原理

2.2.1 Locality

CPU 运行程序是一条条指令地执行的,而且指令地址往往是连续的,即 CPU 在访问内存时,在较短的一段时间内往往集中于某个局部。系统在工作时,可把这些活跃的子程序存入比主存快得多的 cache 中。

- 时间相关性 (Temporal Locality): 如果一个数据现在被访问了,那么以后很有可能也会被访问

- 空间相关性 (Spatial Locality): 如果一个数据现在被访问了,那么它周围的数据在以后可能也会被访问

2.2.2 Hit & Miss

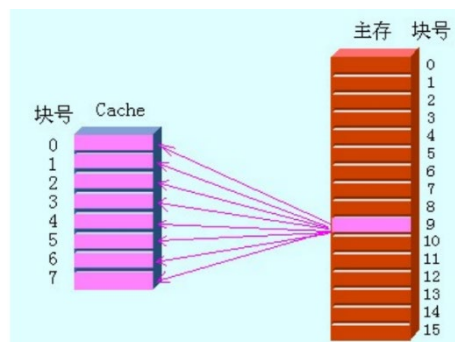
CPU 在访问内存时,首先判断所要访问的内容是否在 cache 中,如果在,则称为命中(hit),此时 CPU 直接从 cache 中调用该内容;否则称为未命中 (miss), CPU 会通过 cache 对主存中的相应内容进行操作

2.3 Cache 映射方式

2.3.1 全相联映射（特殊的组相联）：

主存的任意一块可以映射到 cache 中的任意一块：

- (1) 主存与 cache 分成相同大小的数据块。
- (2) 主存的某一数据块可以装入 cache 的任意一块空间

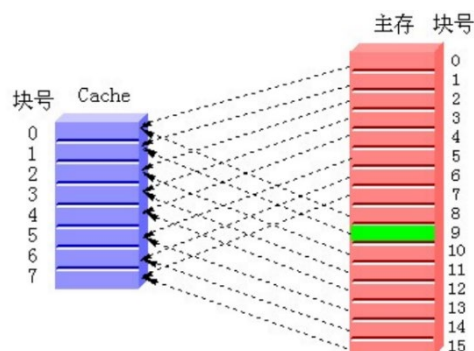


- 优点：命中率比较高，cache 存储空间利用率高。
- 缺点：速度低，成本高，访问相关存储器时，每次都要与全部内容比较。

2.3.2 直接映射：

主存储器中一块只能映射到 cache 的一个特定的块中：

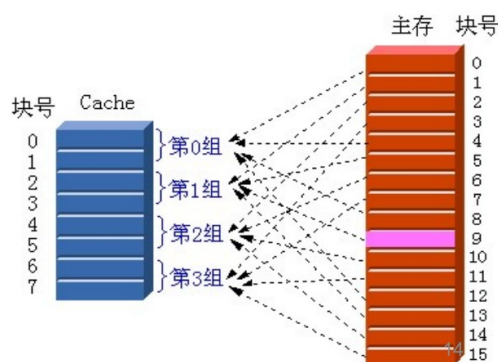
- (1) 主存与 cache 分成相同大小的数据块。
- (2) 主存容量应是 cache 容量的整数倍，将主存空间按 cache 的容量分成区，主存中每一区的块数与 cache 的总块数相等。
- (3) 主存中某区的一块存入 cache 时只能存入缓存中块号相同的位置。



- 优点：地址映射方式简单，数据访问时只需检查区号是否相等即可，因而可以得到比较快的访问速度，硬件设备简单。
- 缺点：替换操作频繁，命中率比较低。

2.3.3 组相联映射（本实验设计）：

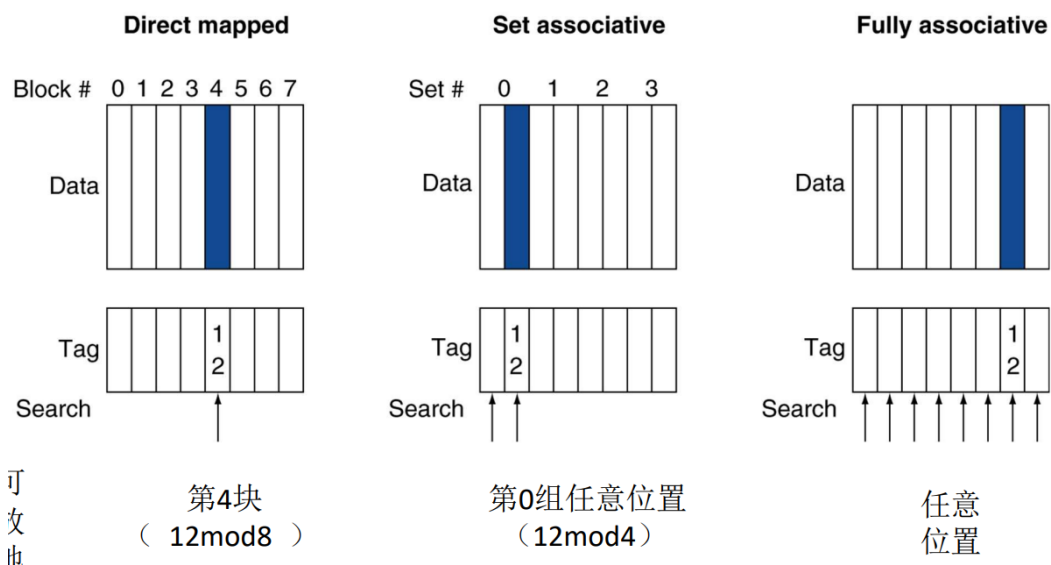
- (1) 主存和 cache 按同样大小划分成块。
- (2) 主存和 cache 按同样大小划分成组。
- (3) 主存容量是 cache 容量的整数倍，将主存空间按 cache 区的大小分成区，主存中每一区的组数与 cache 的组数相同。
- (4) 当主存的数据调入 cache 时，主存与 cache 的组号应相等，也就是各区中的某一块只能存入 cache 的同组号的空间内，但组内各块地址之间则可以任意存放，即从主存的组到 cache 的组之间采用直接映射方式；在两个对应的组内部采用全相联映射方式。



- 优点：块的冲突概率低。块的利用率大幅度提高，块失效率明显降低。
- 缺点：实现难度和造价要比直接映射方式高。

2.3.4 不同映射方式举例：

【12 的放置区域】



2.4 Cache 组织结构

- 块 (block) :块是 cache 与主存的传输单位。
- 路 (way) : 路是组相联映射方式的 cache 结构中的基本存储单位，每一路存储一个块的数据。
- 组 (set/entry) :组是组相联映射方式的 cache 对块进行管理的单位。
- 区 (tag) :块的地址对应的主存储器中的区。
- 块内偏移地址 (offset) : 用来标示块内一个字 (节) 的地址。

组相联映射方式下主存储器的地址空间由区、组和块内偏移地址组成：



2.5 Cache 替换算法

- 先入后出(FILO)算法

- 随机替换(RAND)算法：

随机地确定替换的存储块，设置一个随机数产生器，依据所产生的随机数，确定替换块。这种方法简单、易于实现，但命中率比较低。

- 先入先出(FIFO)算法：

选择那个最先调入的那个块进行替换，当最先调入并被多次命中的块，很可能被优先替换，因而不符合局部性规律。这种方法的命中率比随机法好些，但还不满足要求。

- 近期最少使用(LRU)算法：

依据各块使用的情况，总是选择 那个最近最少使用的块被替换。这种方法比较好地反映了 程序局部性规律，命中率最高。

2.6 Cache 一致性问题

在采用 cache 的系统中，同样一个数据可能既存在于 cache 中（或者只在其一），也存在于主存中，两者数据相同则具有一致性，数据不相同就叫做不一致。

Cache 主要有两种写策略写直达法 (write through) 和**写回法 (write back)**。

- 写直达法 (write through):

在对 cache 进行写操作的同时，也将内容写到主存中。

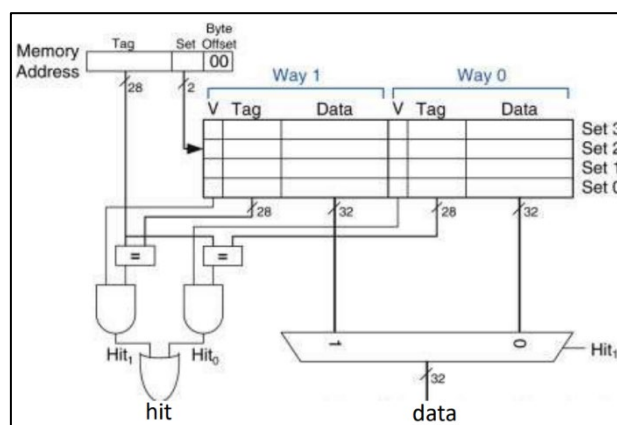
- 优点：可靠性较高，操作过程比较简单。
- 缺点：写操作速度得不到改善，与写主存的速度相同。

- 写回法 (write back):

在 CPU 执行写操作时，只写入 cache，不写入主存。

- 优点：速度较高。
- 缺点：可靠性较差，控制操作比较复杂。

2.6 两路组相联 Cache:



【如图为两路 4 组架构】

2.6.1 Cache line:

Cache line 由 Valid、Tag、Data block 组成:

- Tag: 标签位用于比较在两路中，哪一个 cache 块存了我们的目的数据
- Data block: 存储（内存和 cache 交换的数据块）目标 tag 的数据块。
- valid: 有效位用来表示该 cache 块是否有效。

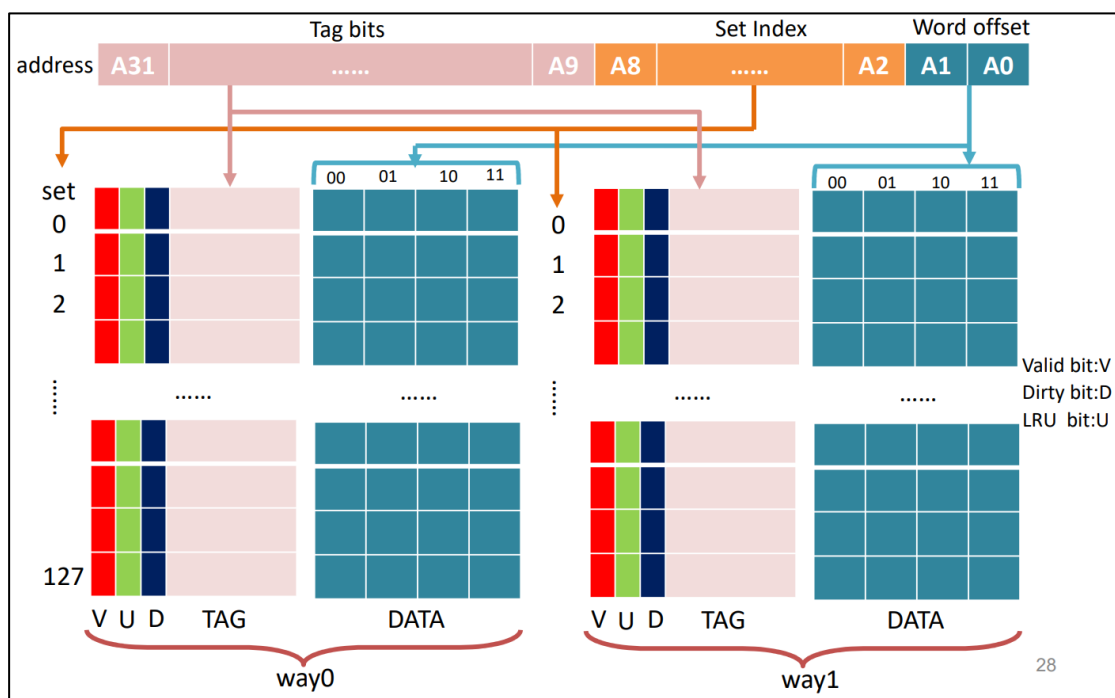
2.6.2 内存地址在 cache 中的映射:



•**Block offset:** 对于内存地址而言，其后 block offset 个字节的数据会构成一个和 cache 做数据交换的块，即 cache 块的大小。

•**index:** 内存地址，其应该被映射到 cache 里的哪一组，故该部分的位数代表的是整个 cache 能容纳多少组；上图，2 路组相联,其中的 MemoryAddress 里的 set 是 2 位，那么这个内存地址就可以根据 set 的两位去找到自己在 cache 中对应的组是哪一组，如:当 set 的两位为 10 时，那么对应到第 3 组 set2。

•**tag:** 使用 index 选出 cache 位于哪一组后去比较具体的 cache 块位于哪一路。



上图是 Lab6 所需设计的内存映射方式以及 cache 的大小，以下为 cache 功能要求：

- ☐ Two-way set associate cache
- ☐ Write Back & Write Allocate
- ☐ Block size is **four** words(16bytes or 128bits)
- ☐ Cache size is **4KiB** -- $2^{12} / 16 = 2^8$ ，因此 index 有 8 位共 128 组
- ☐ Replacement Policy: Least Recently Used

三、实验过程和数据记录

3.1 数据缓存模块的设计：

Cache 主要由两个模块组成：Cache Memory、Cache Controller。

此处将 TAG 和 Data 部分开存储，可以同时并行访问 TAG Ram , Data Ram。

(1) Data_Ram:

此模块来存储 Cache 中 Data 数据，并且能做到读取功能。

```
module Data_ram
#( parameter Index_width = 7,
  parameter Block_width = 128
)
(
  input wire clk, // clock
  input wire en, // enable
  input wire [Index_width-1:0] addr, // address
  input wire [Block_width-1:0] din, // data write in
  output reg [Block_width-1:0] dout // data read out
);

parameter NUM_of_sets = 2 ** Index_width;

reg [Block_width-1:0] cache_data [0:NUM_of_sets-1];
reg [Block_width-1:0] same_data;
// initial begin
//   $readmemh("data_ram0.mem", cache_data);
// end
always @(posedge clk)
begin
  same_data <= cache_data[addr];
  if (en) begin
    cache_data[addr] <= din;
  end
  dout <= same_data;
end
endmodule
```

(2) Tag_Ram:

此模块来存储 Cache 中 Tag 数据，并且能做到修改功能。

```
module Tag_ram
#(  parameter Index_width = 7,
    parameter Tag_width = 23,
    parameter Valid = 1,
    parameter LRU = 1,
    parameter Dirty = 1,
    parameter Block_width = Valid + LRU + Dirty + Tag_width
)
(
    input wire clk, // clock
    input wire en, // enable
    input wire [Index_width-1:0] addr, // address
    input wire [Block_width-1:0] din, // data write in
    output reg [Block_width-1:0] dout // data read out
);

    parameter NUM_of_sets = 2 ** Index_width;

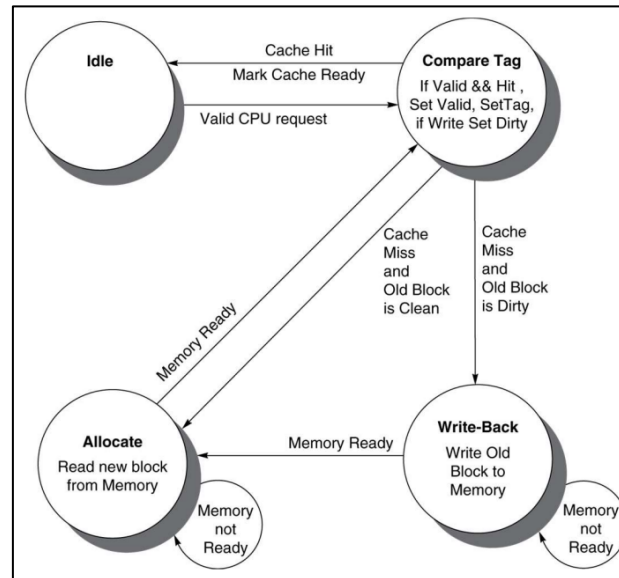
    reg [Block_width-1:0] cache_tag [0:NUM_of_sets-1];
    reg [Block_width-1:0] same_data;

    // initial begin
    //     $readmemh("tag_ram0.mem", cache_tag);
    // end

    always @(posedge clk)
    begin
        same_data <= cache_tag[addr];
        if (en) begin
            cache_tag[addr] <= din;
        end
        dout <= same_data;
    end
endmodule
```

(3) Cache Controller:

Cache 控制采用有限状态机（FSM）实现，简图如下：



• 接口设计:

```
module Cache(
    input wire clk, // clock
    input wire rst, // reset
    input wire [31:0] data_cpu_write, // data write in
    input wire [127:0] data_mem_read, // data read in
    input wire [31:0] addr_cpu, // cpu addr
    input wire wr_cpu, // cpu write enable
    input wire rd_cpu, // cpu read enable
    input wire ready_mem, // memory ready

    output reg wr_mem, // memory write enable
    output reg rd_mem, // memory read enable
    output reg [127:0] data_mem_write, // data to mem
    output reg [31:0] data_cpu_read, // data to cpu
    output reg [31:0] addr_mem // memory addr
);
    reg [1:0] state, next_state;
    parameter [1:0] IDLE = 2'b00, CompareTag = 2'b01, Allocate = 2'b10,
WriteBack = 2'b11;
    reg en0,en1,ent0,ent1; // input enable
    reg [127:0] wdata; // input din for Data_ram
    reg [25:0] wtag0, wtag1; // input din for Tag_ram
    wire [6:0] index; // input addr for ram
    wire [22:0] cpu_tag; // Tag in addr_cpu
    wire [1:0] cpu_offset;
```

```

wire [127:0] rdata0, rdata1;
wire [25:0] rtag0, rtag1;

wire cpu_req_valid;
wire hit0, hit1, hit;

wire valid0, valid1, valid;
wire LRU0, LRU1;
wire dirty0, dirty1;

assign cpu_req_valid = wr_cpu | rd_cpu;
assign index = addr_cpu[8:2];
assign cpu_tag = addr_cpu[31:9];
assign cpu_offset = addr_cpu[1:0];
assign valid0 = rtag0[25];
assign LRU0 = rtag0[24];
assign dirty0 = rtag0[23];

assign valid1 = rtag1[25];
assign LRU1 = rtag1[24];
assign dirty1 = rtag1[23];

assign hit0 = (cpu_tag == rtag0[22:0]) & valid0;
assign hit1 = (cpu_tag == rtag1[22:0]) & valid1;
assign hit = hit0 | hit1;
assign valid = valid0 | valid1;
assign dirty = dirty0 | dirty1;

```

- Memory 模块导入:

```

Data_ram0 data_ram0(
    .clk(~clk),
    .en(en0),//
    .din(wdata),//
    .addr(index),
    .dout(rdata0)//
);
Data_ram1 data_ram1(
    .clk(~clk),
    .en(en1),//
    .din(wdata),//
    .addr(index),
    .dout(rdata1)//
);

```

```

Tag_ram0 tag_ram0(
    .clk(~clk),
    .en(ent0),//
    .din(wtag0),//
    .addr(index),
    .dout(rtag0)//
);
Tag_ram1 tag_ram1(
    .clk(~clk),
    .en(ent1),//
    .din(wtag1),//
    .addr(index),
    .dout(rtag1)//
);

```

状态机分为 4 个状态：

(1) IDLE 准备：

```

IDLE: begin
    en0  = 1'b0;
    en1  = 1'b0;
    ent0 = 1'b0;
    ent1 = 1'b0;
    wr_mem = 1'b0;
    rd_mem = 1'b0;

    if(cpu_req_valid)
        next_state = CompareTag;
    else
        next_state = IDLE;
end

```

(2) CompareTag 比较 tag:

```

CompareTag: begin
    en0  = 1'b0;
    en1  = 1'b0;
    ent0 = 1'b0;
    ent1 = 1'b0;
    wr_mem = 1'b0;
    rd_mem = 1'b0;
    if(hit) begin
        next_state = IDLE;
        ent0 = 1'b1;
    end
end

```

```

ent1 = 1'b1;
if(rd_cpu) begin
    if(hit0) begin
        wtag0 = {rtag0[25],1'b0,rtag0[23:0]}; // change LRU (0
for new)
        wtag1 = {rtag1[25],1'b1,rtag1[23:0]}; // change LRU (1
for old)

        case(cpu_offset)
            2'b00: data_cpu_read = rdata0[31:0];
            2'b01: data_cpu_read = rdata0[63:32];
            2'b10: data_cpu_read = rdata0[95:64];
            2'b11: data_cpu_read = rdata0[127:96];
        endcase
    end
    if(hit1) begin
        wtag0 = {rtag0[25],1'b1,rtag0[23:0]}; // change LRU (0
for old)
        wtag1 = {rtag1[25],1'b0,rtag1[23:0]}; // change LRU (1
for new)

        case(cpu_offset)
            2'b00: data_cpu_read = rdata1[31:0];
            2'b01: data_cpu_read = rdata1[63:32];
            2'b10: data_cpu_read = rdata1[95:64];
            2'b11: data_cpu_read = rdata1[127:96];
        endcase
    end
end
if(wr_cpu) begin
    if(hit0) begin
        en0 = 1'b1;
        wtag0 = {rtag0[25],1'b0,1'b1,rtag0[22:0]}; // change LRU
(0 for new), change dirty (1 for only in cache)
        wtag1 = {rtag1[25],1'b1,rtag1[23:0]}; // change LRU (1
for old)

        case(cpu_offset)
            2'b00: wdata = {rdata0[127:32],data_cpu_write};
            2'b01: wdata =
{rdata0[127:64],data_cpu_write,rdata0[31:0]};
            2'b10: wdata =
{rdata0[127:96],data_cpu_write,rdata0[63:0]};
            2'b11: wdata = {data_cpu_write,rdata0[95:0]};
        endcase
    end
end

```

```

        else if(hit1) begin
            en1 = 1'b1;
            wtag0 = {rtag0[25],1'b1,rtag0[23:0]}; // change LRU (1
for old)
            wtag1 = {rtag1[25],1'b0,1'b1,rtag1[23:0]}; // change LRU
(0 for new), change dirty (1 for only in cache)

            case(cpu_offset)
                2'b00: wdata = {rdata1[127:32],data_cpu_write};
                2'b01: wdata =
{rdata1[127:64],data_cpu_write,rdata1[31:0]};
                2'b10: wdata =
{rdata1[127:96],data_cpu_write,rdata1[63:0]};
                2'b11: wdata = {data_cpu_write,rdata1[95:0]};
            endcase
        end
    end
end
else begin
    if(dirty)
        next_state = WriteBack;
    else
        next_state = Allocate;
    end
end
end

```

(3) Allocate 块分配:

```

Allocate: begin
    en0 = 1'b0;
    en1 = 1'b0;
    ent0 = 1'b0;
    ent1 = 1'b0;
    rd_mem = 1'b0;
    wr_mem = 1'b0;
    addr_mem = {addr_cpu[31:2],2'b00};
    wdata = data_mem_read;
    if(ready_mem) begin
        next_state = CompareTag;
        rd_mem = 1'b1; // memory read enable
        ent0 = 1'b1;
        ent1 = 1'b1;
        if(LRU0) begin
            en0 = 1'b1;

```

```

        wtag0 = {1'b1,1'b0,1'b0,cpu_tag}; // valid , LRU (0 for
new), dirty (0 for both in cache and memory)
        wtag1 = {1'b1,1'b1,rtag1[23:0]}; // valid, LRU (1 for old)
    end
    else begin
        en1 = 1'b1;
        wtag0 = {1'b1,1'b1,rtag0[23:0]};
        wtag1 = {1'b1,1'b0,1'b0,cpu_tag};
    end
end
else
    next_state = Allocate;
end
end

```

(4) WriteBack 内存写回:

```

WriteBack: begin
    en0 = 1'b0;
    en1 = 1'b0;
    ent0 = 1'b0;
    ent1 = 1'b0;
    rd_mem = 1'b0;
    wr_mem = 1'b0;
    addr_mem = {addr_cpu[31:2],2'b00};
    if(ready_mem) begin
        next_state = Allocate;
        wr_mem = 1'b1; // memory write enable
        if(dirty0)
            data_mem_write = rdata0;
        else if(dirty1)
            data_mem_write = rdata1;
    end
    else
        next_state = WriteBack;
end
end

```

(5) 阶段更新部分:

```

always@(posedge clk or posedge rst) begin
    if(rst) begin
        state <= IDLE;
    end
    else begin
        state <= next_state;
    end
end
end

```


3.2 数据缓存模块仿真验证:

仿真代码如下, 结果分析见第肆部分:

```
`timescale 1ns / 1ps
module Cache_tb();
    reg clk;
    reg rst;
    reg [31:0] addr_cpu;
    reg rd_cpu;
    reg wr_cpu;
    reg ready_mem;

    // Outputs
    wire [31:0] addr_mem;
    wire rd_mem;
    wire wr_mem;

    // Bidirs
    reg [31:0] data_cpu_write;
    reg [127:0] data_mem_read;
    wire [127:0] data_mem_write;
    wire [31:0] data_cpu_read;

    // Instantiate the Unit Under Test (UUT)
    Cache uut (
        .clk(clk),
        .rst(rst),
        .data_cpu_write(data_cpu_write),
        .data_mem_read(data_mem_read),
        .addr_cpu(addr_cpu),
        .addr_mem(addr_mem),
        .rd_cpu(rd_cpu),
        .wr_cpu(wr_cpu),
        .rd_mem(rd_mem),
        .wr_mem(wr_mem),
        .data_cpu_read(data_cpu_read),
        .data_mem_write(data_mem_write),
        .ready_mem(ready_mem)
    );

    //Clock generator block
    initial begin
        clk = 1'd0;
    forever
```

```

        #10 clk = ~clk;
    end

    initial begin
        // Initialize Inputs
        rst = 1;
        addr_cpu = 0;
        rd_cpu = 0;
        wr_cpu = 0;
        ready_mem = 1;
        data_cpu_write = 0;
        data_mem_read = 0;

        rst = 0;

        #120;

        // Read from location
        rd_cpu = 1'd1;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_0111;
        #40;
        rd_cpu = 1'd0;

        // Write to same location
        #40;
        wr_cpu = 1'd1;
        data_cpu_write = 32'habababab;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_0111;
        #40;
        wr_cpu = 1'd0;
        // Read from same location to check updated data
        #40;
        rd_cpu = 1'd1;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_0111;
        #40;
        rd_cpu = 1'd0;

        // // Read Miss with dirty bit 0 policy check, reads data from
        // Main Memory

        #40;
        rd_cpu = 1'd1;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_1010; // read
        miss
    end

```

```

        #40;

        ready_mem = 1;
        data_mem_read = 128'h54088666_54088666_54088666_54088666; //
miss

        #40;
        rd_cpu = 1'd0;
        #40;
        rd_cpu = 1'd1;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_1010;
        #40;
        rd_cpu = 1'd0;

// ////////////////////////////////////////
// Write Miss with dirty bit 1 policy check
        #80;
        rd_cpu = 1'd0;
        wr_cpu = 1'd1;
        data_cpu_write = 32'h5a5a5a5a;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_1101;
        #40;

        ready_mem = 1;
        data_mem_read = 128'h11451450_11451451_11451452_11451453;

        #40;

        data_cpu_write = 32'h5a5a5a5a;
        addr_cpu = 32'b0000_0000_0000_0000_0000_0010_0000_1101;
        #40;
        wr_cpu = 1'd0;

        #300 $finish;
    end

endmodule

```

四、实验结果分析

```
initial begin
    cache_data[0] = 128'h00000001_00000002_00000003_00000004;
    cache_data[1] = 128'h10000001_10000002_10000003_10000004;
    cache_data[2] = 128'h20000001_20000002_20000003_20000004;
    cache_data[3] = 128'h30000001_30000002_30000003_30000004;
end // Data_ram0

initial begin
    cache_data[0] = 128'h0000000a_0000000b_0000000c_0000000d;
    cache_data[1] = 128'h1000000a_1000000b_1000000c_1000000d;
    cache_data[2] = 128'h2000000a_2000000b_2000000c_2000000d;
    cache_data[3] = 128'h3000000a_3000000b_3000000c_3000000d;
end // Data_ram1

initial begin
    cache_tag[0] = 26'b10000000000000000000000000;
    cache_tag[1] = 26'b10000000000000000000000001;
    cache_tag[2] = 26'b10000000000000000000000010;
    cache_tag[3] = 26'b10000000000000000000000011;
end //Tag_ram0

initial begin
    cache_tag[0] = 26'b1000000000000000000000001000;
    cache_tag[1] = 26'b1000000000000000000000001001;
    cache_tag[2] = 26'b1000000000000000000000001010;
    cache_tag[3] = 26'b1110000000000000000000001011; // valid but
[LRU(old) + dirty]
end // Tag_ram1
```

对 cache memory 进行初始化，依次为 Data0（Way0）、Tag0（Way0）、Data1（Way1）Tag1（Way1）。

(1) Read Hit:



```
addr_cpu = 32'b0000_0000_0000_0000_0000_001_0000001_11; = 207
```

——> Index = 0000001 第一组

——> Tag = 000001 和 rtag0 相等 (hit)

——> 读出 rdata0 的第 b'11 = d'3 个字的数据 (10000001)。

(2) Write hit:



同一个 addr_cpu, 现在想要写入 32h'abababab, 可以看到 rdata0 中的数据被修改 (Write hit), But (data_cpu_write 没被截进来)。

随后对同一地址进行 Read (rd_cpu 使能), 可以看到 data_cpu_red 成功读出 abababab, 即 Read hit。

(3) Read Miss (No dirty): 采用 allocate 从内存中读取块



```
addr_cpu = 32'b0000_0000_0000_0000_0000_001_0000010_10; = 20a
```

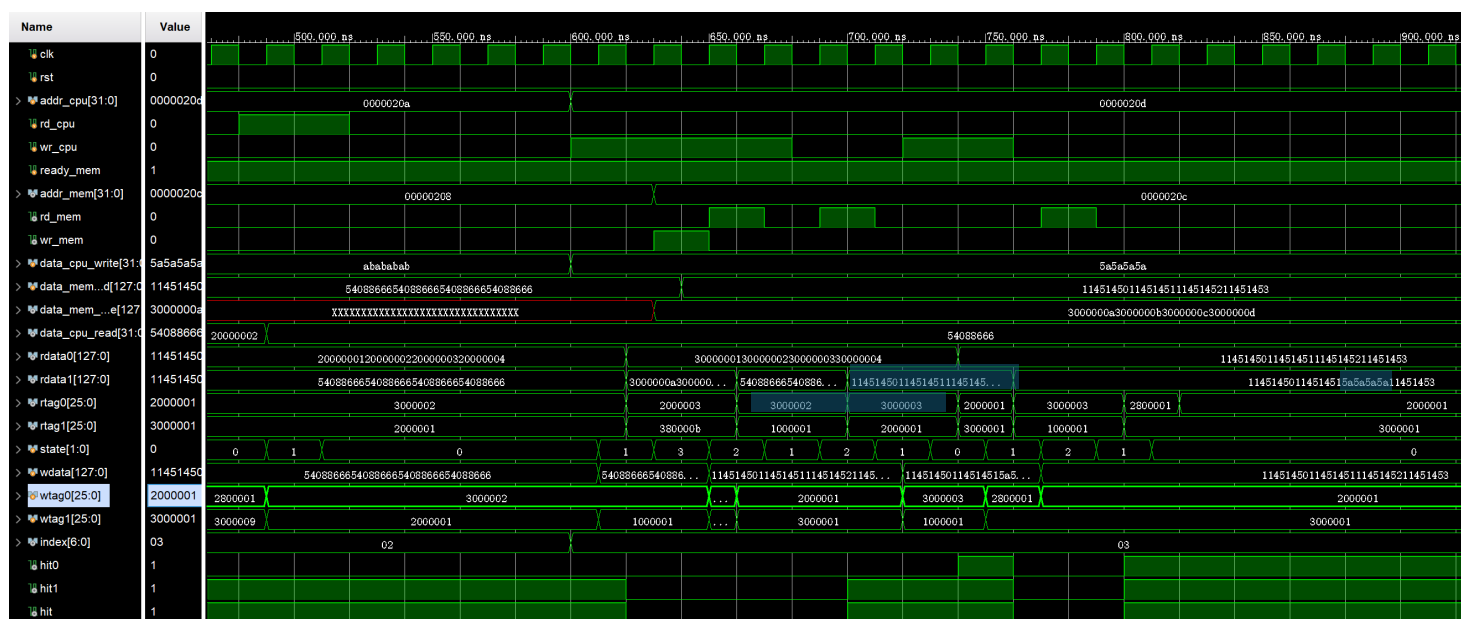
——> Index = 0000010 第二组

——> Tag = 000001 不与 cache 中相应块中任意路的 tag 相等 (miss)

——> 从 CompareTag(state = 1) 进入 Allocate (state=2), 因为块 not dirty (rtag1 的第 23 位不为 1) 不用写回。

——> 随后 rdata1 中修改, 且 rtag0 设为 LRU=1 (old) ——> 随后 Read hit

(4) Write Miss (Dirty): 先 WriteBack 写回, 随后 Allocate 从内存中读取块



```
addr_cpu = 32'b0000_0000_0000_0000_0000_001_0000011_01; = 20a
```

——> Index = 0000011 第三组

——> Tag = 000001 不与 cache 相应块的 tag 相等 (writemiss)

——> 从 CompareTag(state = 1) 进入 WB (state=3) (rtag1 块 dirty) 写回。

——> 进入 Allocate(state = 2), 并修改 rtag1, 并且 rdata1 写入

——> 然后在同一位置写入 5a5a5a5a, 可以看到被修改 (write hit)

五、讨论与心得

Cache 的实现逻辑还是蛮清晰的, 但是有些阶段间跳转以及 valid、LRU、Dirty 位的修正等细节需要比较注意。

终章~~~